SET 3 | 45 Q

Sample Questions

Full-Length Set 3

Databricks Certified Data Engineer Associate (exam version May 4, 2026)

- 45 new questions (no overlap with Set 1, Set 2, or Remedial)
- Heavier on gaps: D1 2 | D2 9 | D3 9 | D4 6 | D5 6 | D6 7 | D7 6
- Objective line quoted verbatim from the exam outline
- Answer key with explanations, distractor analysis, and doc URLs
- Claims checked against docs.databricks.com (Sep 2026)

Study-only material. Not affiliated with Databricks. Real exam items are not disclosed.

How to Use This Set

This set adds weight to areas that had less practice in Sets 1 and 2: Git folders, Spark UI diagnosis, JDBC, REST, and federated ingestion, and Unity Catalog privilege details (DENY, volumes, managed storage). Each question copies the structure of the official exam guide samples: an **Objective** line quoted from the exam outline, a short scenario with concrete constraints, a one-sentence question, and four options (A to D) with exactly one correct answer. Answers and explanations are collected at the end, as in the exam guide, so you can attempt the questions without seeing them.

Section	Domain	Exam weight	Questions	Range
Section 1	Databricks Intelligence Platform	6%	2	Q1-Q2
Section 2	Data Ingestion and Loading	21%	9	Q3-Q11
Section 3	Data Transformation and Modeling	22%	9	Q12-Q20
Section 4	Working with Lakeflow Jobs	16%	6	Q21-Q26
Section 5	Implementing CI/CD	10%	6	Q27-Q32
Section 6	Troubleshooting, Monitoring, and Optimization	10%	7	Q33-Q39
Section 7	Governance and Security	15%	6	Q40-Q45

EXAM FOCUS Pacing: 45 scored questions in 90 minutes is about **2 minutes per question**. Time yourself on this set at the same pace.

TIP Pattern from the official samples: the correct option is usually the **Databricks-native, managed, or automatic** feature. Distractors tend to be (1) workable but sub-optimal manual fixes, (2) legacy or anti-pattern approaches, (3) swapped details such as formats, defaults, or limits, and (4) a real product applied to the wrong use case.

NOTE Product names follow the current exam guide: Lakeflow Jobs, Lakeflow Connect, Lakeflow Spark Declarative Pipelines, and Declarative Automation Bundles (formerly Databricks Asset Bundles).

SECTION 1 Databricks Intelligence Platform

Exam weight 6%

Question 1

Objective: *Understand Databricks Data Intelligence Platform's compute services, including their characteristics, limitations, and cost models, and select the most suitable option for each workload use case.*

A data science team needs a classic cluster to run R notebooks and some legacy code that uses RDD APIs. Their current cluster uses **standard** access mode, and those workloads fail.

Which change should the data engineer make?

- **A.** Switch the compute to dedicated access mode
- **B.** Increase the number of workers on the standard cluster
- **C.** Move the workloads to a serverless SQL warehouse
- **D.** Enable Photon on the standard cluster

Question 2

Objective: *Understand the core components of the Databricks Data Intelligence Platform, such as its architecture, Delta Lake, and Unity Catalog.*

A security reviewer asks where workloads physically run in an AWS Databricks deployment that uses both classic job clusters and serverless notebooks.

Which statement is correct?

- **A.** Both classic and serverless compute run in the customer's AWS account
- **B.** Both classic and serverless compute run in the Databricks control plane
- **C.** Classic compute runs in the customer's AWS account; serverless compute runs in a compute layer in the Databricks account, in the same region as the workspace
- **D.** Serverless compute runs in the customer's AWS account; classic compute runs in the Databricks account

SECTION 2 Data Ingestion and Loading

Exam weight 21%

Question 3

Objective: *Use JDBC/ODBC or REST clients in notebooks to land data into cloud storage or directly into Unity Catalog-governed tables, usually orchestrated and scheduled with Lakeflow Jobs.*

A notebook calls a vendor REST API with a bearer token and lands the JSON responses in Unity Catalog. A code review finds the token hard-coded in the notebook, where anyone with view access can read it.

What is the recommended way to handle the token?

- **A.** Store it in a notebook widget default value
- **B.** Store it in a Databricks secret scope, read it with `dbutils.secrets.get(scope, key)`, and control access with secret ACLs
- **C.** Encode it in base64 inside the notebook
- **D.** Put it in a Delta table that only the team can read

Question 4

Objective: *Enable and detail data ingestion patterns, including batch, streaming, and incremental loading, and import data from sources such as local files, Lakeflow Connect standard connectors, and Lakeflow Connect managed connectors.*

Analysts want to run occasional ad hoc reports against an operational PostgreSQL database from Databricks SQL, with Unity Catalog governance, **without copying the data** into the lakehouse.

Which approach fits best?

- **A.** Lakehouse Federation, with a connection and a foreign catalog for the PostgreSQL database
- **B.** A Lakeflow Connect managed connector that replicates every table
- **C.** A nightly JDBC export to CSV files followed by Auto Loader
- **D.** COPY INTO from the PostgreSQL server

Question 5

Objective: *Enable and detail data ingestion patterns, including batch, streaming, and incremental loading, and import data from sources such as local files, Lakeflow Connect standard connectors, and Lakeflow Connect managed connectors.*

A Structured Streaming job reads from Kafka with `format("kafka")`, `kafka.bootstrap.servers`, and `subscribe`. The bronze table shows the `value` column as unreadable bytes, although the producers send JSON text.

What should the data engineer do?

- **A.** Set `startingOffsets` to `earliest`
- **B.** Switch to `subscribePattern`
- **C.** Cast the `value` column to a string, for example `CAST(value AS STRING)`, and then parse the JSON
- **D.** Increase the number of Kafka partitions

Question 6

Objective: *Enable and detail data ingestion patterns, including batch, streaming, and incremental loading, and import data from sources such as local files, Lakeflow Connect standard connectors, and Lakeflow Connect managed connectors.*

A team must ingest data from a SaaS application that has no Lakeflow Connect managed connector. They want a vendor-managed ingestion service that can be set up from the Databricks UI and automatically provisions the Databricks resources it needs.

Which option fits this requirement?

- **A.** Lakehouse Federation
- **B.** Delta Sharing
- **C.** A dashboard task
- **D.** An ingestion partner such as Fivetran through Partner Connect

---

Question 7

Objective: *Configure Lakeflow Connect to reliably ingest data from diverse enterprise sources into Unity Catalog-governed tables.*

A data engineer configures a Lakeflow Connect managed connector for a SaaS source and wants to know what kind of Unity Catalog objects the pipeline produces, so that downstream jobs can be designed correctly.

Which objects does a managed connector pipeline write to?

- **A.** Streaming tables
- **B.** Temporary views
- **C.** External volumes
- **D.** Materialized views only

Question 8

Objective: *Use Auto Loader with schema enforcement and schema evolution in batch modes (for example, directory listing or file notification) to land data into Unity Catalog-governed tables.*

An Auto Loader stream reads CSV files without a provided schema. After the first run, every column in the bronze table, including `price` and `quantity`, has the type STRING.

Which option makes Auto Loader infer proper numeric types?

- **A.** `cloudFiles.schemaEvolutionMode = rescue`
- **B.** `cloudFiles.useNotifications = true`
- **C.** `cloudFiles.inferColumnTypes = true`
- **D.** `mergeSchema = true` on the write

Question 9

Objective: *Use Auto Loader with schema enforcement and schema evolution in batch modes (for example, directory listing or file notification) to land data into Unity Catalog-governed tables.*

Auto Loader infers the schema of JSON files correctly, except that `order_ts` should be TIMESTAMP and `amount` should be DECIMAL(10,2). The team wants to keep automatic inference for all other columns and does not want to write out a full schema.

Which option should be used?

- **A.** Provide the complete schema with `.schema()`
- **B.** `cloudFiles.schemaHints = "order_ts TIMESTAMP, amount DECIMAL(10,2)"`
- **C.** `cloudFiles.inferColumnTypes = false`
- **D.** `cloudFiles.schemaEvolutionMode = failOnNewColumns`

Question 10

Objective: *Use JDBC/ODBC or REST clients in notebooks to land data into cloud storage or directly into Unity Catalog-governed tables, usually orchestrated and scheduled with Lakeflow Jobs.*

A notebook loads data from a REST API into Unity Catalog every hour. The raw JSON responses must be kept as files for replay and auditing, and the bronze table must be loaded incrementally from those files.

Which design follows Databricks patterns?

- **A.** Keep the responses in a Python list on the driver and append them to the bronze table at the end of the day
- **B.** Write the responses as JSON files to a Unity Catalog volume path such as `/Volumes/main/raw/api_landing/`, then ingest them with Auto Loader into the bronze table
- **C.** Write the responses to the DBFS root and query them with a view
- **D.** Insert each response directly into a materialized view

---

Question 11

Objective: *Use JDBC/ODBC or REST clients in notebooks to land data into cloud storage or directly into Unity Catalog-governed tables, usually orchestrated and scheduled with Lakeflow Jobs.*

To speed up a JDBC read from a production MySQL database, an engineer sets `numPartitions = 400` on a large cluster. Soon after, the database team reports that the source server is overloaded.

What does Databricks guidance recommend?

- **A.** Keep 400 and add more cluster workers
- **B.** Remove `partitionColumn` so that Spark picks partitions automatically
- **C.** Set `fetchsize` to 1
- **D.** Keep `numPartitions` moderate, generally not above about 50, because too many simultaneous queries can degrade the remote database

---

SECTION 3 Data Transformation and Modeling

Exam weight 22%

Question 12

Objective: *Perform data deduplication operations and aggregate operations on DataFrames, such as count, approximate count distinct, and mean, summary.*

The bronze table `customer_updates` has several rows per `customer_id`. The silver table must keep only the row with the latest `updated_at` for each customer.

Which PySpark approach returns exactly that row?

- **A.** `df.dropDuplicates(["customer_id"])`
- **B.** `df.groupBy("customer_id").agg(F.max("updated_at"))`
- **C.** `df.distinct()`
- **D.** Add `row_number()` over `Window.partitionBy("customer_id").orderBy(F.desc("updated_at"))` and keep rows where it equals 1

Question 13

Objective: *Combine DataFrames with operations such as Inner join, left join, broadcast join, multiple keys, cross join, union, and union all.*

`orders` and `shipments` must be joined on both `order_id` and `line_no`. The result must contain each join key column only once.

Which join expression achieves this?

- **A.** `orders.join(shipments, ["order_id", "line_no"], "inner")`
- **B.** `orders.join(shipments, orders.order_id == shipments.order_id)`
- **C.** `orders.crossJoin(shipments)`
- **D.** `orders.join(shipments, "order_id").join(shipments, "line_no")`

Question 14

Objective: *Combine DataFrames with operations such as Inner join, left join, broadcast join, multiple keys, cross join, union, and union all.*

A Gold report must show sales for **every** store on **every** calendar date, including zero-sale days. The engineer has a `stores` DataFrame (200 rows) and a `calendar` DataFrame (365 rows).

Which operation builds the complete store-by-date grid before a left join to sales?

- **A.** `stores.union(calendar)`
- **B.** `stores.crossJoin(calendar)`
- **C.** `stores.join(calendar, "store_id", "left_anti")`
- **D.** `stores.join(calendar, "store_id")`

Question 15

Objective: *Perform data deduplication operations and aggregate operations on DataFrames, such as count, approximate count distinct, and mean, summary.*

Before building silver tables, an engineer wants a quick profile of the numeric columns in a DataFrame: count, mean, standard deviation, min, the 25th, 50th, and 75th percentiles, and max.

Which single DataFrame method returns all of these by default?

- **A.** `df.count()`
- **B.** `df.agg(F.mean("*"))`
- **C.** `df.summary()`
- **D.** `df.approxQuantile()`

Question 16

Objective: *Understand the basic tuning parameters (spark.sql.shuffle.partitions, spark.default.parallelism, spark.executor/driver.memory, spark.sql.autoBroadcastJoinThreshold) and re-measure the performance.*

After the dimension tables grew, a job started failing with executor memory errors. The Spark UI shows a broadcast hash join on a dimension that is now several GB, chosen automatically because the engineer had raised `spark.sql.autoBroadcastJoinThreshold`.

Which configuration change stops automatic broadcasting so the join falls back to a shuffle join? (Re-measure afterwards.)

- **A.** Set `spark.sql.autoBroadcastJoinThreshold = -1`
- **B.** Set `spark.sql.shuffle.partitions = 1`
- **C.** Add `broadcast()` hints to every join
- **D.** Raise `spark.sql.autoBroadcastJoinThreshold` further

Question 17

Objective: *Implement data cleaning by reading bronze tables with PySpark/SQL, cleaning nulls, standardizing data types, and writing to new silver tables.*

A daily MERGE upserts `updates` into `silver.customers` on `customer_id`. One day it fails with an error that multiple source rows matched the same target row.

What is the correct fix?

- **A.** Swap the source and target in the MERGE
- **B.** Remove the `WHEN MATCHED` clause
- **C.** Deduplicate the source so that each `customer_id` appears once (for example, keep the latest row), then run the MERGE
- **D.** Switch the MERGE to `INSERT OVERWRITE`

Question 18

Objective: *Implement data cleaning by reading bronze tables with PySpark/SQL, cleaning nulls, standardizing data types, and writing to new silver tables.*

A silver table must exactly mirror a daily full snapshot from a source system: matching rows are updated, new rows are inserted, and rows that no longer exist in the snapshot are **deleted**.

Which MERGE clauses implement this?

- **A.** `WHEN MATCHED THEN DELETE` only
- **B.** `WHEN NOT MATCHED THEN INSERT *` only
- **C.** `WHEN MATCHED THEN UPDATE SET *` together with `INSERT OVERWRITE`
- **D.** `WHEN MATCHED THEN UPDATE SET *`, `WHEN NOT MATCHED THEN INSERT *`, and `WHEN NOT MATCHED BY SOURCE THEN DELETE`

Question 19

Objective: *Apply data quality checks and validation rules to ensure reliable Silver and Gold datasets.*

In a Lakeflow Spark Declarative Pipeline, a CDC feed of customer changes can arrive out of order. The silver table must keep the full history of address changes, with validity periods.

Which definition meets this requirement?

- **A.** `AUTO CDC INTO silver_customers FROM stream(cdc_feed) KEYS (customer_id) SEQUENCE BY event_ts STORED AS SCD TYPE 2`
- **B.** `AUTO CDC INTO silver_customers FROM stream(cdc_feed) KEYS (customer_id) STORED AS SCD TYPE 1`
- **C.** `CREATE MATERIALIZED VIEW silver_customers AS SELECT * FROM cdc_feed`
- **D.** A MERGE that keeps the last row received for each `customer_id`

Question 20

Objective: *Apply data quality checks and validation rules to ensure reliable Silver and Gold datasets.*

A data engineer runs `ALTER TABLE silver.orders ADD CONSTRAINT positive_amt CHECK (amount > 0)` on an existing table, and the command fails.

What is the most likely cause?

- **A.** CHECK constraints are supported only on external tables
- **B.** Some existing rows already have `amount <= 0`, and Databricks checks every existing row before adding the constraint
- **C.** CHECK constraints require a materialized view
- **D.** The table must be partitioned on `amount`

SECTION 4 Working with Lakeflow Jobs Exam weight 16%

Question 21

Objective: *Configure common tasks (notebook, SQL query, dashboard, and pipeline tasks) and their dependencies using Lakeflow Jobs and its DAG-based task graph*

Three teams each own an ingestion job. A master job must run all three, in a controlled order, before a shared Gold job, while still reusing the existing jobs without copying their tasks.

Which task type should the master job use?

- **A.** Run Job tasks that trigger each existing job
- **B.** Notebook tasks that copy the code from each job
- **C.** For each task over the job names
- **D.** Dashboard tasks

Question 22

Objective: *Configure common tasks (notebook, SQL query, dashboard, and pipeline tasks) and their dependencies using Lakeflow Jobs and its DAG-based task graph*

A job defines the job parameter `run_date`, which every task uses. An operator needs to reprocess 2026-09-15 once, manually, without editing the job definition.

What should the operator do?

- **A.** Edit the job definition, run it, and then change it back
- **B.** Use *Run now with different parameters* and set `run_date` to 2026-09-15
- **C.** Create a copy of the job with the new date
- **D.** Set a task value named `run_date` in the first task

Question 23

Objective: *Implement job schedules using Lakeflow Jobs with an understanding of trigger types (scheduled, file arrival, and table update)*

An hourly job sometimes runs longer than one hour. Maximum concurrent runs is left at the default, and queueing is **disabled**. At 10:00 the 09:00 run is still going.

What happens to the 10:00 scheduled run?

- **A.** It starts in parallel with the 09:00 run
- **B.** It cancels the 09:00 run
- **C.** It is skipped, because the default maximum of 1 concurrent run has been reached
- **D.** It is held until the 09:00 run finishes

Question 24

Objective: *Implement control flows (retries and conditional tasks such as branching and looping) using Lakeflow Jobs for pipeline orchestration*

A 10-task job run failed at task 7 because of a bad parameter. Tasks 1–6 succeeded and took 3 hours. The parameter has been fixed in the job settings.

What is the most efficient way to complete the run?

- **A.** Start a new run of the whole job
- **B.** Repair the run, which re-runs only the unsuccessful tasks and their dependents using the current settings
- **C.** Delete tasks 1–6 from the job and run it again
- **D.** Increase retries on task 7 and wait for the next schedule

---

Question 25

Objective: *Implement job schedules using Lakeflow Jobs with an understanding of trigger types (scheduled, file arrival, and table update)*

A Structured Streaming job must always be running. If it fails, it should restart automatically with backoff, and there should never be more than one instance running.

Which trigger type should be used?

- **A.** A scheduled trigger every minute
- **B.** A file arrival trigger
- **C.** A table update trigger
- **D.** A continuous trigger

Question 26

Objective: *Choose between time-based and data-driven triggers based on data availability and pipeline dependencies.*

A Gold job reads both `silver.orders` and `silver.customers`. It should start only after **both** tables have been updated, not when just one of them changes.

How should the trigger be configured?

- **A.** A table update trigger on both tables, set to fire when all monitored tables are updated
- **B.** A table update trigger on `silver.orders` only
- **C.** Two separate jobs, each with a trigger on one table
- **D.** A file arrival trigger on the tables' storage paths

SECTION 5 Implementing CI/CD Exam weight 10%

Question 27

Objective: *Manage your code development workflow within the Databricks workspace UI, including creating and switching between branches in Databricks Git Folders (formerly Databricks Repos), committing and pushing changes, and creating pull requests using Databricks Git integration.*

A teammate merged changes into the remote `dev` branch in GitHub. A developer's Databricks Git folder is checked out on `dev` but still shows the old notebook code.

What should the developer do in the Git dialog?

- **A.** Click *Pull*
- **B.** Click *Commit & Push*
- **C.** Reset the branch
- **D.** Create a new branch from `dev`

Question 28

Objective: *Manage your code development workflow within the Databricks workspace UI, including creating and switching between branches in Databricks Git Folders (formerly Databricks Repos), committing and pushing changes, and creating pull requests using Databricks Git integration.*

During a pull in a Databricks Git folder, a merge conflict appears in `transform.py`. The developer wants to keep their own version of the file and discard the incoming change.

How can this be resolved in the Databricks UI?

- **A.** Delete the Git folder and clone it again
- **B.** Commit the file with the conflict markers still in it
- **C.** Choose *Keep all current changes* for the file, or edit it manually, and then complete the merge
- **D.** Conflicts can be resolved only in the Git provider

Question 29

Objective: *Manage your code development workflow within the Databricks workspace UI, including creating and switching between branches in Databricks Git Folders (formerly Databricks Repos), committing and pushing changes, and creating pull requests using Databricks Git integration.*

A developer has uncommitted edits on `feature-a` and switches the Git folder to `feature-b`. The edits do not conflict with the code on `feature-b`.

What happens to the uncommitted edits?

- **A.** They are discarded
- **B.** They are automatically committed to `feature-a`
- **C.** They are stashed and hidden until the developer switches back
- **D.** They carry over and appear as uncommitted changes on `feature-b`

Question 30

Objective: *Manage your code development workflow within the Databricks workspace UI, including creating and switching between branches in Databricks Git Folders (formerly Databricks Repos), committing and pushing changes, and creating pull requests using Databricks Git integration.*

Notebooks are stored in a Git folder in source format (`.py`). A reviewer asks why the pull request contains no cell outputs or charts from the developer's last run.

What explains this?

- **A.** The developer forgot to save the notebook
- **B.** When notebooks are saved in source formats such as `.py`, `.sql`, `.scala`, or `.r`, outputs are not included in commits; the IPYNB format is needed to commit outputs
- **C.** GitHub strips outputs automatically
- **D.** Outputs are committed only on the `main` branch

Question 31

Objective: *Understand environment-specific configuration using Automation Bundle (formerly Databricks Asset Bundles) variables and overrides while promoting the same codebase across dev, test, and prod targets.*

A bundle's `databricks.yml` has grown to 900 lines because it defines 30 jobs. The team wants each job in its own YAML file, and the prod target must deploy to a different workspace URL than dev.

Which configuration achieves this?

- **A.** Create 30 separate bundles, one per job
- **B.** Use `variables` to hold the job definitions
- **C.** Use the `include` mapping for the resource YAML files, and set `workspace.host` under the prod target
- **D.** Put the prod workspace URL in each job's notebook

Question 32

Objective: *Understand the Databricks CLI to validate, deploy, and manage Declarative Automation Bundles (formerly Databricks Asset Bundles) and other workspace assets in automated CI/CD workflows.*

A GitHub Actions pipeline runs `databricks bundle deploy -t prod`. It currently authenticates with a senior engineer's personal access token, which is stored as a repository secret.

What does Databricks recommend instead?

- **A.** Rotate the engineer's personal access token monthly
- **B.** Share one workspace admin password among CI pipelines
- **C.** Commit a `.databrickscfg` file to the repository
- **D.** Authenticate as a service principal using OAuth, preferably with workload identity federation

---

SECTION 6 Troubleshooting, Monitoring, and Optimization

Exam weight 10%

Question 33

Objective: *Identify common performance bottlenecks such as data skew, shuffling, and disk spilling by interpreting stage-level metrics in the Spark UI.*

In the Spark UI, a join stage's Summary Metrics show task durations of 25th percentile 40 s, median 45 s, 75th percentile 50 s, and **Max 14 min**. The stage details show no significant spill.

What is the most likely problem, and what should be checked first?

- **A.** Data skew; confirm that adaptive query execution's skew join handling is active, or address the skewed key
- **B.** Disk spill; increase the number of shuffle partitions
- **C.** Too many small files; run `OPTIMIZE`
- **D.** Driver overload; increase the driver size

Question 34

Objective: *Identify common performance bottlenecks such as data skew, shuffling, and disk spilling by interpreting stage-level metrics in the Spark UI.*

A scan stage reads 1.2 TB in 400 seconds on a cluster with 1,000 worker cores. Most queries filter on `customer_id` and `event_date`, but the table has no data layout optimization.

What does the Spark UI guide suggest about this stage, and what helps most?

- **A.** It is CPU bound; enable more shuffle partitions
- **B.** It is skewed; salt the keys
- **C.** It is spilling; use memory-optimized instances
- **D.** It is roughly I/O bound (about 3 MB per second per core); reduce the data read, for example with liquid clustering on the filter columns so that files can be skipped

Question 35

Objective: *Identify common performance bottlenecks such as data skew, shuffling, and disk spilling by interpreting stage-level metrics in the Spark UI.*

A job on all-purpose compute takes 50 minutes, but the Spark UI jobs timeline shows Spark jobs running for only about 15 minutes, separated by long gaps. The notebook runs a Python `for` loop that calls a pandas function on each file.

What do the gaps most likely mean?

- **A.** The executors are spilling to disk
- **B.** Non-Spark code is running on the driver; rewrite the loop using Spark so that the work is distributed
- **C.** Shuffle partitions are set too high
- **D.** The cluster is waiting for the autoscaler to remove nodes

Question 36

Objective: *Diagnose cluster startup failures, library conflicts, and out-of-memory issues.*

One all-purpose cluster runs about 40 concurrent Structured Streaming queries. The cluster metrics show the driver under heavy load while the workers are mostly idle, and the queries are falling behind.

What does Databricks recommend trying first?

- **A.** Add more workers
- **B.** Increase `spark.sql.shuffle.partitions`
- **C.** Double the size of the driver, then consider reducing concurrency or spreading the streams across clusters
- **D.** Disable adaptive query execution

Question 37

Objective: *Diagnose cluster startup failures, library conflicts, and out-of-memory issues.*

A job fails intermittently with `ExecutorLostFailure (executor 4 exited caused by one of the running tasks)`. The team suspects memory but is not sure, because the error is generic.

Which diagnostic step does the Spark UI guide recommend?

- **A.** Switch to a worker type with double the memory per core and see whether the job fails later or not at all
- **B.** Halve the number of workers
- **C.** Turn off Photon
- **D.** Increase the retry count to 10

Question 38

Objective: *Understand the features of Liquid Clustering and predictive optimization.*

A workspace has predictive optimization enabled. Its managed tables are compacted and vacuumed automatically, but a large **external** table in the same schema keeps accumulating small files and old file versions.

Why, and what should the team do?

- **A.** Predictive optimization runs only weekly; wait a few more days
- **B.** Predictive optimization applies only to Unity Catalog managed tables; schedule `OPTIMIZE` and `VACUUM` for the external table, or convert it to managed
- **C.** External tables need `CLUSTER BY AUTO` to opt in
- **D.** Predictive optimization must be enabled separately on every external location

---

Question 39

Objective: *Use the Lakeflow Jobs UI to monitor pipeline health by interpreting job statuses, viewing DAG-based task graphs to spot upstream blockers, and tracking pipeline run times and failure rates.*

A job has a task `notify_failure` with *Run if* set to *At least one failed*, downstream of `load_a` and `load_b`. In last night's run, both loads succeeded, and the task graph shows `notify_failure` with a status that is not Failed or Succeeded.

Which status is expected, and what does it mean?

- **A.** *Upstream failed*, meaning a dependency failed
- **B.** *Timed out*, meaning the task took too long
- **C.** *Excluded*, meaning its Run if condition was not met, so it intentionally did not run
- **D.** *Skipped*, meaning the maximum concurrent runs limit was reached

---

SECTION 7 Governance and Security Exam weight 15%

Question 40

Objective: *Configure access controls using the UI and SQL by applying GRANT, REVOKE, and DENY privileges to principals (users, groups, and service principals) at appropriate levels of the security hierarchy.*

An engineer runs `DENY SELECT ON TABLE main.finance.payroll TO contractors` in a Unity Catalog workspace to make sure contractors can never read the table, and the statement is not supported.

Which statement is correct?

- **A.** The DENY SQL statement applies only to the legacy `hive_metastore`; in Unity Catalog, remove access by revoking the SELECT grant and checking for grants inherited from the schema or catalog
- **B.** DENY works only on schemas, not tables
- **C.** DENY requires the MANAGE privilege
- **D.** DENY must be run from a SQL warehouse instead of a cluster

---

Question 41

Objective: *Configure access controls using the UI and SQL by applying GRANT, REVOKE, and DENY privileges to principals (users, groups, and service principals) at appropriate levels of the security hierarchy.*

An ML team needs to read images from the volume `main.ml.raw_images` and also upload new image files to it.

Which privileges must they have on the volume, in addition to USE CATALOG and USE SCHEMA?

- **A.** SELECT and MODIFY
- **B.** READ FILES and WRITE FILES
- **C.** READ VOLUME only
- **D.** READ VOLUME and WRITE VOLUME

Question 42

Objective: *Differentiate between managed and external tables in Unity Catalog and perform basic operations (create, modify, delete, and convert between managed and external tables) on them.*

The metastore has a managed storage location, and the catalog `finance` was created with its own `MANAGED LOCATION`. The schema `finance.ap` has no managed location. A new managed table is created in `finance.ap`.

Where are the table's data files stored?

- **A.** In the metastore's managed location
- **B.** In the `finance` catalog's managed location
- **C.** In the workspace's DBFS root
- **D.** The CREATE TABLE fails because the schema has no location

---

Question 43

Objective: *Configure access controls using the UI and SQL by applying GRANT, REVOKE, and DENY privileges to principals (users, groups, and service principals) at appropriate levels of the security hierarchy.*

A production job runs as a service principal and must read from `main.gold.revenue`. A personal user account must not be used.

Which grants give the service principal least-privilege read access?

- **A.** Add the service principal to the workspace admins group
- **B.** Grant USE CATALOG, USE SCHEMA, and SELECT to the service principal, identified by its application ID in backticks
- **C.** Grant SELECT to the job owner's personal account
- **D.** Grant ALL PRIVILEGES on the catalog to the service principal

---

Question 44

Objective: *Understand column-level masking and row-level security to restrict data visibility based on user groups.*

`hr.salaries` has a row filter and a column mask. A data engineer tries to create a test copy with `SHALLOW CLONE` and to query the table with `VERSION AS OF`, and both operations fail.

What explains this?

- **A.** Clones and time travel require the table to be external
- **B.** Tables with row filters or column masks do not support deep or shallow clones, or time travel
- **C.** The engineer needs the MODIFY privilege
- **D.** Row filters must be suspended with `VACUUM` first

Question 45

Objective: *Understand column-level masking and row-level security to restrict data visibility based on user groups.*

Business users in the group `sales_mgrs` should see a curated, joined result of `orders` and `customers` with emails hidden, but must **not** be granted access to the base tables.

Which approach fits this requirement?

- **A.** A dynamic view over the base tables that hides emails using `is_account_group_member()`, with SELECT granted to `sales_mgrs` on the view only
- **B.** Grant SELECT on both base tables and trust users to avoid the email column
- **C.** Make a shallow clone of each table without the email column
- **D.** Put the joined data in a volume as CSV files

ANSWERS

Answer Key

Answers and explanations

#	Answer	Section
1	**A**	D1
2	**C**	D1
3	**B**	D2
4	**A**	D2
5	**C**	D2
6	**D**	D2
7	**A**	D2
8	**C**	D2
9	**B**	D2
10	**B**	D2
11	**D**	D2
12	**D**	D3
13	**A**	D3
14	**B**	D3
15	**C**	D3
16	**A**	D3
17	**C**	D3
18	**D**	D3
19	**A**	D3
20	**B**	D3
21	**A**	D4
22	**B**	D4
23	**C**	D4
24	**B**	D4
25	**D**	D4
26	**A**	D4
27	**A**	D5
28	**C**	D5
29	**D**	D5
30	**B**	D5
31	**C**	D5
32	**D**	D5
33	**A**	D6
34	**D**	D6
35	**B**	D6
36	**C**	D6
37	**A**	D6
38	**B**	D6
39	**C**	D6
40	**A**	D7
41	**D**	D7
42	**B**	D7
43	**B**	D7
44	**B**	D7
45	**A**	D7

REVIEW **Explanations** Rationale, distractors, and sources

Question 1 Section 1

Answer: A Switch the compute to dedicated access mode

Why: Standard access mode (formerly shared) does not support RDD APIs, R, or Databricks Runtime for ML. The docs say to use dedicated compute (formerly single user) when a workload needs these features.

Distractors: B and D: performance changes do not remove the feature limits. C: SQL warehouses run SQL, not R or RDD code.

Source:
https://docs.databricks.com/aws/en/compute/access-mode-limitations

Question 2 Section 1

Answer: C Classic compute runs in the customer's AWS account; serverless compute runs in a compute layer in the Databricks account, in the same region as the workspace

Why: The control plane holds the backend services that Databricks manages. Data is processed in the compute plane: classic compute runs in the customer's AWS account in a customer-managed network, and serverless compute runs in the Databricks account in the workspace's region.

Distractors: A, B, and D each place one of the compute types in the wrong account or plane.

Source:
https://docs.databricks.com/aws/en/getting-started/high-level-architecture

Question 3 Section 2

Answer: B Store it in a Databricks secret scope, read it with `dbutils.secrets.get(scope, key)`, and control access with secret ACLs

Why: Secrets are read with `dbutils.secrets.get`, and their values are replaced with `[REDACTED]` when displayed. Access is controlled with ACLs on the secret scope. Redaction applies only to the literal value, so a transformed secret can still be exposed.

Distractors: A and C: still visible in the notebook. D: a table is not a secret store and gives no redaction.

Source:
https://docs.databricks.com/aws/en/security/secrets/

Question 4 Section 2

Answer: A Lakehouse Federation, with a connection and a foreign catalog for the PostgreSQL database

Why: Lakehouse Federation gives governed, read-only access to external data through Unity Catalog foreign catalogs, with query pushdown and no data movement. The docs list ad hoc reporting, BI, and proof-of-concept access as typical uses. When higher data volumes and lower latency matter more, Databricks recommends Lakeflow Connect instead.

Distractors: B and C: both copy the data. D: COPY INTO loads files from cloud storage, not databases.

Source:
https://docs.databricks.com/aws/en/query-federation/

Question 5 Section 2

Answer: C Cast the `value` column to a string, for example `CAST(value AS STRING)`, and then parse the JSON

Why: Kafka `key` and `value` are always deserialized as byte arrays. They must be cast or deserialized explicitly, for example with `cast("string")` followed by JSON parsing. The source also returns `topic`, `partition`, `offset`, and `timestamp`.

Distractors: A: changes where the stream starts reading, not the data type. B: changes how topics are selected. D: unrelated to the format.

Source:
https://docs.databricks.com/aws/en/connect/streaming/kafka

Question 6 Section 2

Answer: D An ingestion partner such as Fivetran through Partner Connect

Why: Partner Connect connects a workspace to partner solutions from the Databricks UI and provisions resources such as SQL warehouses, service principals, and tokens for the partner. Fivetran and Rivery are listed among the ingestion partners. Partner Connect requires the Premium plan or higher.

Distractors: A: federation is for querying external databases, not SaaS ingestion. B: Delta Sharing shares data between platforms. C: a dashboard task refreshes dashboards.

Source:
https://docs.databricks.com/aws/en/partner-connect/

Question 7 Section 2

Answer: A Streaming tables

Why: The docs describe managed connector pipelines as querying the source and writing the results to streaming tables. These pipelines are governed by Unity Catalog and run on serverless compute.

Distractors: B: session-scoped and not persisted. C: volumes hold files, not ingested tables. D: materialized views are for computed results.

Source:
https://docs.databricks.com/aws/en/ingestion/lakeflow-connect/

Question 8 Section 2

Answer: C `cloudFiles.inferColumnTypes = true`

Why: For JSON and CSV, Auto Loader infers every column as a string by default, including nested JSON fields. Setting `cloudFiles.inferColumnTypes` to `true` makes it infer types from sample data.

Distractors: A: controls how schema changes are handled. B: controls file discovery. D: controls schema evolution on the write, not type inference.

Source:
https://docs.databricks.com/aws/en/ingestion/cloud-object-storage/auto-loader/schema

Question 9 Section 2

Answer: B `cloudFiles.schemaHints = "order_ts TIMESTAMP, amount DECIMAL(10,2)"`

Why: Schema hints override the inferred types of specific columns, written as a string in SQL schema syntax. Auto Loader uses schema hints only when no schema is provided, so every other column is still inferred.

Distractors: A: requires writing out every column. C: makes inference worse. D: governs how new columns are handled.

Source:
https://docs.databricks.com/aws/en/ingestion/cloud-object-storage/auto-loader/schema

Question 10 Section 2

Answer: B Write the responses as JSON files to a Unity Catalog volume path such as `/Volumes/main/raw/api_landing/`, then ingest them with Auto Loader into the bronze table

Why: Volumes govern non-tabular files under Unity Catalog and can be written from Python through `/Volumes/<catalog>/<schema>/<volume>/` paths. Auto Loader then ingests new files incrementally, with checkpointed exactly-once tracking, and the raw files stay available for replay.

Distractors: A: data is lost if the driver fails, and the files are not kept. C: the DBFS root is not governed by Unity Catalog. D: materialized views are defined by queries, not by inserts.

Source:
https://docs.databricks.com/aws/en/volumes/
https://docs.databricks.com/aws/en/ingestion/cloud-object-storage/auto-loader/

Question 11 Section 2

Answer: D Keep `numPartitions` moderate, generally not above about 50, because too many simultaneous queries can degrade the remote database

Why: `numPartitions` sets how many concurrent queries are sent to the source database. The docs suggest matching it to the cluster's cores, for example 8 for 8 cores, and caution against values above about 50 because that many simultaneous queries can hurt the remote database's performance.

Distractors: A: makes the load worse. B: goes back to a single-threaded read. C: makes the read far slower.

Source:
https://docs.databricks.com/aws/en/connect/external-systems/jdbc

Question 12 Section 3

Answer: D Add `row_number()` over `Window.partitionBy("customer_id").orderBy(F.desc("updated_at"))` and keep rows where it equals 1

Why: A window with `row_number()` ordered by `updated_at` descending ranks each customer's rows deterministically, so filtering on 1 keeps the latest full row.

Distractors: A: keeps an arbitrary row, not necessarily the latest. B: returns only the key and the max timestamp, and loses the other columns. C: removes only rows that are identical in every column.

Source:
https://spark.apache.org/docs/latest/api/python/reference/pyspark.sql/api/pyspark.sql.functions.row_number.html
https://spark.apache.org/docs/latest/api/python/reference/pyspark.sql/api/pyspark.sql.Window.html

Question 13 Section 3

Answer: A `orders.join(shipments, ["order_id", "line_no"], "inner")`

Why: Passing a list of column names joins on all of those columns and returns each key column only once. A join on an expression keeps both sides' key columns.

Distractors: B: matches on only one key and duplicates `order_id`. C: a Cartesian product. D: joins twice, which multiplies rows.

Source:
https://spark.apache.org/docs/latest/api/python/reference/pyspark.sql/api/pyspark.sql.DataFrame.join.html

Question 14 Section 3

Answer: B `stores.crossJoin(calendar)`

Why: A cross join returns the Cartesian product, here 200 × 365 = 73,000 rows, which gives every store-date combination. Left joining sales to that grid produces rows for zero-sale days.

Distractors: A: stacks rows instead of combining them. C and D: the calendar has no `store_id` column to join on.

Source:
https://spark.apache.org/docs/latest/api/python/reference/pyspark.sql/api/pyspark.sql.DataFrame.crossJoin.html

Question 15 Section 3

Answer: C `df.summary()`

Why: By default, `summary()` computes count, mean, stddev, min, 25%, 50%, 75%, and max. You can also pass the statistics you want.

Distractors: A: returns only the row count. B: not valid usage, and returns only means. D: returns only quantiles.

Source:
https://spark.apache.org/docs/latest/api/python/reference/pyspark.sql/api/pyspark.sql.DataFrame.summary.html

Question 16 Section 3

Answer: A Set `spark.sql.autoBroadcastJoinThreshold = -1`

Why: `spark.sql.autoBroadcastJoinThreshold` is the maximum size of a table that Spark broadcasts automatically (10 MB by default), and setting it to `-1` disables automatic broadcasting. The Databricks Spark UI guide lists a large broadcast as one cause of memory problems. Explicit `broadcast()` hints still force a broadcast, so remove those too if they exist.

Distractors: B: a single partition causes extreme spill. C and D: make the problem worse.

Source:
https://spark.apache.org/docs/latest/sql-performance-tuning.html
https://docs.databricks.com/aws/en/optimizations/spark-ui-guide/spark-memory-issues

Question 17 Section 3

Answer: C Deduplicate the source so that each `customer_id` appears once (for example, keep the latest row), then run the MERGE

Why: A MERGE can fail when several source rows match and try to update the same target row, because the update is ambiguous. The docs say to preprocess the source to remove the possibility of multiple matches.

Distractors: A: wrong semantics. B: stops updating existing customers. D: replaces the table instead of upserting.

Source:
https://docs.databricks.com/aws/en/delta/merge

Question 18 Section 3

Answer: D `WHEN MATCHED THEN UPDATE SET *`, `WHEN NOT MATCHED THEN INSERT *`, and `WHEN NOT MATCHED BY SOURCE THEN DELETE`

Why: `WHEN NOT MATCHED BY SOURCE` runs for target rows that have no matching source row, so adding `THEN DELETE` removes records that are missing from the snapshot. Combined with update and insert, this keeps the target in sync with the snapshot.

Distractors: A and B: each handles only part of the sync. C: mixes two incompatible write patterns.

Source:
https://docs.databricks.com/aws/en/delta/merge

Question 19 Section 3

Answer: A `AUTO CDC INTO silver_customers FROM stream(cdc_feed) KEYS (customer_id) SEQUENCE BY event_ts STORED AS SCD TYPE 2`

Why: SCD Type 2 keeps a history of records, with `__START_AT` and `__END_AT` columns marking each version. `SEQUENCE BY` orders events by their sequence value rather than arrival time, so late-arriving events do not overwrite newer states. In Python the equivalent is `create_auto_cdc_flow()`.

Distractors: B: Type 1 keeps no history, and the missing SEQUENCE BY means ordering is not handled. C: no CDC semantics. D: arrival order is wrong when events come out of order.

Source:
https://docs.databricks.com/aws/en/ldp/cdc

Question 20 Section 3

Answer: B Some existing rows already have `amount <= 0`, and Databricks checks every existing row before adding the constraint

Why: Before adding a NOT NULL or CHECK constraint, Databricks verifies that all existing rows satisfy it. After the constraint is added, any write that violates it fails. Fix or remove the violating rows first.

Distractors: A, C, and D are not requirements.

Source:
https://docs.databricks.com/aws/en/tables/constraints

Question 21 Section 4

Answer: A Run Job tasks that trigger each existing job

Why: The Run Job task triggers other jobs as tasks, which makes orchestration modular and reusable. Job parameters are pushed down and can be overridden. Databricks limits nesting to three levels of Run Job tasks and does not allow circular calls.

Distractors: B: duplicates code. C: iterates over inputs for one nested task rather than orchestrating different jobs. D: refreshes dashboards.

Source:
https://docs.databricks.com/aws/en/jobs/run-job

Question 22 Section 4

Answer: B Use *Run now with different parameters* and set `run_date` to 2026-09-15

Why: Job parameters are defined at the job level and pushed down to every task. A user with CAN MANAGE RUN or higher can override them for a manual run with *Run now with different parameters*.

Distractors: A and C: unnecessary changes. D: task values are produced at runtime and do not override job parameters.

Source:
https://docs.databricks.com/aws/en/jobs/parameters

Question 23 Section 4

Answer: C It is skipped, because the default maximum of 1 concurrent run has been reached

Why: New jobs default to a maximum of 1 concurrent run, and Databricks skips a run that would exceed the limit. With queueing enabled, which is the default for jobs created in the UI after April 15, 2024, runs are instead held in a queue for up to 48 hours.

Distractors: A: would need max concurrent runs greater than 1. B: not the behavior. D: describes queueing, which is disabled here.

Source:
https://docs.databricks.com/aws/en/jobs/advanced

Question 24 Section 4

Answer: B Repair the run, which re-runs only the unsuccessful tasks and their dependents using the current settings

Why: Repair run re-runs only the failed or canceled tasks and the tasks that depend on them, and skips tasks that already succeeded. Unsuccessful tasks are re-run with the current job and task settings, so the fixed parameter takes effect.

Distractors: A: repeats 3 hours of work. C: destructive. D: delays the fix until the next scheduled run.

Source:
https://docs.databricks.com/aws/en/jobs/repair-job-failures

Question 25 Section 4

Answer: D A continuous trigger

Why: Databricks recommends continuous mode for always-on streaming workloads. Only one instance runs at a time, and failures are retried with exponential backoff. Continuous jobs cannot use task dependencies or retry policies.

Distractors: A: overlapping runs, then skips. B and C: data-driven triggers that start runs, not keep one running.

Source:
https://docs.databricks.com/aws/en/jobs/continuous

Question 26 Section 4

Answer: A A table update trigger on both tables, set to fire when all monitored tables are updated

Why: A table update trigger can monitor several tables and fire when any one of them is updated or only when all of them have been updated. Choosing the all-tables condition enforces the dependency on both tables.

Distractors: B: ignores the update to customers. C: duplicates work and still does not wait for both. D: file arrival triggers watch locations and volumes, not table commits.

Source:
https://docs.databricks.com/aws/en/jobs/trigger-table-update

Question 27 Section 5

Answer: A Click *Pull*

Why: *Pull* updates the notebooks and files in the Git folder to the latest version in the remote repository.

Distractors: B: sends local changes to the remote and does not fetch new ones. C: replaces the branch contents and history, so local work is lost. D: does not refresh `dev`.

Source:
https://docs.databricks.com/aws/en/repos/git-operations-with-repos

Question 28 Section 5

Answer: C Choose *Keep all current changes* for the file, or edit it manually, and then complete the merge

Why: The Git folder UI lets you resolve conflicts by editing the file manually or by choosing *Keep all current changes* or *Take all incoming changes*.

Distractors: A: loses local work. B: commits a broken file. D: the Databricks UI supports conflict resolution.

Source:
https://docs.databricks.com/aws/en/repos/git-operations-with-repos

Question 29 Section 5

Answer: D They carry over and appear as uncommitted changes on `feature-b`

Why: When you switch branches, uncommitted changes on the current branch carry over and appear as uncommitted changes on the new branch, as long as they do not conflict with the code there.

Distractors: A, B, and C do not match the documented behavior.

Source:
https://docs.databricks.com/aws/en/repos/git-operations-with-repos

Question 30 Section 5

Answer: B When notebooks are saved in source formats such as `.py`, `.sql`, `.scala`, or `.r`, outputs are not included in commits; the IPYNB format is needed to commit outputs

Why: Notebook outputs are not included in commits by default when notebooks are saved in source file formats. Committing outputs requires the IPYNB format.

Distractors: A, C, and D do not explain this behavior.

Source:
https://docs.databricks.com/aws/en/repos/git-operations-with-repos

Question 31 Section 5

Answer: C Use the `include` mapping for the resource YAML files, and set `workspace.host` under the prod target

Why: `databricks.yml` can pull in other configuration files, such as resource files, through the `include` mapping. Targets can override settings such as the workspace `host` while inheriting the job definitions from the top level.

Distractors: A: loses shared configuration and promotion. B: variables are for values, not resource definitions. D: mixes deployment configuration into code.

Source:
https://docs.databricks.com/aws/en/dev-tools/bundles/settings

Question 32 Section 5

Answer: D Authenticate as a service principal using OAuth, preferably with workload identity federation

Why: For CI/CD, Databricks recommends service principals instead of users' personal access tokens. OAuth token federation (workload identity federation) is the most secure option because no Databricks secret needs to be stored. OAuth M2M with `DATABRICKS_HOST`, `DATABRICKS_CLIENT_ID`, and `DATABRICKS_CLIENT_SECRET` is another option.

Distractors: A: still a personal token. B and C: insecure practices.

Source:
https://docs.databricks.com/aws/en/dev-tools/ci-cd/

Question 33 Section 6

Answer: A Data skew; confirm that adaptive query execution's skew join handling is active, or address the skewed key

Why: The Spark UI guide flags skew when the Max task duration is much higher than the 75th percentile, about 50% or more. Here Max is roughly 17 times the 75th percentile. AQE splits skewed partitions in shuffle-based joins when a partition is more than 5 times the median and larger than 256 MB.

Distractors: B: there is no spill. C and D: do not match a single slow task.

Source:
https://docs.databricks.com/aws/en/optimizations/spark-ui-guide/long-spark-stage-page
https://docs.databricks.com/aws/en/optimizations/aqe

Question 34 Section 6

Answer: D It is roughly I/O bound (about 3 MB per second per core); reduce the data read, for example with liquid clustering on the filter columns so that files can be skipped

Why: The guide's test divides the I/O size by total worker cores and then by duration. A result around 3 MB per second per core suggests the stage is I/O bound: 1.2 TB ÷ 1,000 cores ÷ 400 s ≈ 3 MB/s. Remedies include liquid clustering for better data skipping, filtering early, Photon, and dynamic file pruning.

Distractors: A, B, and C do not fit the metrics.

Source:
https://docs.databricks.com/aws/en/optimizations/spark-ui-guide/long-spark-stage-io

Question 35 Section 6

Answer: B Non-Spark code is running on the driver; rewrite the loop using Spark so that the work is distributed

Why: The guide says that any code that is not Spark shows up in the timeline as a gap, for example Python loops with native functions. Other causes include idle time on all-purpose compute, plan compilation on the driver, and an overloaded driver.

Distractors: A and C: would show up inside Spark jobs, not as gaps between them. D: not a documented cause.

Source:
https://docs.databricks.com/aws/en/optimizations/spark-ui-guide/spark-job-gaps

Question 36 Section 6

Answer: C Double the size of the driver, then consider reducing concurrency or spreading the streams across clusters

Why: A common cause of an overloaded driver is too many concurrent streams, queries, or jobs on one cluster. Databricks recommends first doubling the driver size and checking the effect, then reducing concurrency or spreading the load across several clusters.

Distractors: A: the workers are already idle. B and D: do not reduce load on the driver.

Source:
https://docs.databricks.com/aws/en/optimizations/spark-ui-guide/spark-driver-overloaded

Question 37 Section 6

Answer: A Switch to a worker type with double the memory per core and see whether the job fails later or not at all

Why: The guide notes that executor-lost errors are generic and recommends doubling the memory per core as a test. If the job takes longer to fail or stops failing, memory is the issue. Listed causes include too few shuffle partitions, large broadcasts, UDFs, window functions without `PARTITION BY`, skew, and streaming state.

Distractors: B, C, and D do not diagnose a memory problem.

Source:
https://docs.databricks.com/aws/en/optimizations/spark-ui-guide/spark-memory-issues

Question 38 Section 6

Answer: B Predictive optimization applies only to Unity Catalog managed tables; schedule `OPTIMIZE` and `VACUUM` for the external table, or convert it to managed

Why: Predictive optimization automatically runs `OPTIMIZE`, `VACUUM`, and `ANALYZE` only on Unity Catalog managed tables, and not on external tables. Converting the table with `SET MANAGED` enables predictive optimization automatically, unless it was turned off manually.

Distractors: A, C, and D are not how predictive optimization works.

Source:
https://docs.databricks.com/aws/en/optimizations/predictive-optimization
https://docs.databricks.com/aws/en/tables/convert-to-managed

Question 39 Section 6

Answer: C *Excluded*, meaning its Run if condition was not met, so it intentionally did not run

Why: For *At least one failed* and *All failed*, a task whose condition is not met is marked Excluded, which is expected behavior and not an error. For conditions such as *All succeeded*, an unmet condition produces Upstream failed instead.

Distractors: A: no dependency failed. B and D: unrelated statuses.

Source:
https://docs.databricks.com/aws/en/jobs/run-if

Question 40 Section 7

Answer: A The DENY SQL statement applies only to the legacy `hive_metastore`; in Unity Catalog, remove access by revoking the SELECT grant and checking for grants inherited from the schema or catalog

Why: The `DENY` statement is supported only for `hive_metastore` objects, where a denial takes precedence over any grant. Unity Catalog does not support it. In Unity Catalog, access comes only from grants, and privileges granted on a parent apply to all children, so the parent levels must be checked too. Unity Catalog's ABAC DENY policies (Beta) can currently deny only `MANAGE ACCESS CONTROL`.

Distractors: B, C, and D all misstate why the statement fails.

Source:
https://docs.databricks.com/aws/en/sql/language-manual/security-deny
https://docs.databricks.com/aws/en/data-governance/unity-catalog/access-control/permissions-concepts
https://docs.databricks.com/aws/en/data-governance/unity-catalog/abac/deny-policies

Question 41 Section 7

Answer: D READ VOLUME and WRITE VOLUME

Why: `READ VOLUME` allows reading and listing files. `WRITE VOLUME`, together with READ VOLUME, is required to create, update, or delete files. All volume access also requires USE CATALOG and USE SCHEMA on the parents.

Distractors: A: these are table privileges. B: privileges on external locations, not volumes. C: read-only.

Source:
https://docs.databricks.com/aws/en/volumes/privileges

Question 42 Section 7

Answer: B In the `finance` catalog's managed location

Why: The most specific location wins: the schema's managed location if it has one, otherwise the catalog's, otherwise the metastore's. Catalog and schema locations are set with `MANAGED LOCATION` or changed with `ALTER ... SET MANAGED LOCATION`.

Distractors: A: the catalog's location takes precedence over the metastore's. C: not used for Unity Catalog managed tables. D: a schema location is not required.

Source:
https://docs.databricks.com/aws/en/connect/unity-catalog/cloud-storage/managed-storage

Question 43 Section 7

Answer: B Grant USE CATALOG, USE SCHEMA, and SELECT to the service principal, identified by its application ID in backticks

Why: Service principals are granted privileges like any other principal, identified by their application ID in backticks. Reading a table requires USE CATALOG, USE SCHEMA, and SELECT.

Distractors: A and D: far more than needed. C: ties production access to a person.

Source:
https://docs.databricks.com/aws/en/data-governance/unity-catalog/manage-privileges/
https://docs.databricks.com/aws/en/data-governance/unity-catalog/manage-privileges/privileges

Question 44 Section 7

Answer: B Tables with row filters or column masks do not support deep or shallow clones, or time travel

Why: The documented limitations for row filters and column masks include no time travel, no deep or shallow clones, and filter or mask functions that cannot reference other tables that also have active filters or masks. Row filters and masks also require Databricks Runtime 12.2 LTS or above.

Distractors: A, C, and D do not explain these documented limitations.

Source:
https://docs.databricks.com/aws/en/tables/row-and-column-filters

Question 45 Section 7

Answer: A A dynamic view over the base tables that hides emails using `is_account_group_member()`, with SELECT granted to `sales_mgrs` on the view only

Why: Dynamic views wrap one or more base tables in a view that filters rows, masks columns, or reshapes data, usually gated by group-membership functions such as `is_account_group_member()`. They fit cases where curated data is exposed to specific groups without granting access to the underlying tables.

Distractors: B: exposes the raw data. C: copies data and adds governance overhead. D: not governed table access.

Source:
https://docs.databricks.com/aws/en/tables/row-and-column-filters

WARNING Q12-Q15 (PySpark window, join, cross join, and `summary()` behavior) and Q16 (`autoBroadcastJoinThreshold = -1`) cite the Apache Spark API and tuning references, because docs.databricks.com does not document these individually. All other answers are grounded in docs.databricks.com, checked Oct 4, 2026.